

Assignment 7

Shahriar Hasan

Id: 2010776115

October 18, 2025

Objective

This assignment covers the following three tasks:

1. **Object Detection:** Detect objects in a face-free video using YOLOv8, YOLOv11, and YOLOv12 models. Compare their performance based on speed (FPS).
2. **Face Detection:** Fine-tune a YOLOv8 model using the WIDER FACE dataset and evaluate its performance.
3. **Model Comparison:** Compare the fine-tuned YOLOv8 face detector with the YOLOv8-Face model from GitHub using the WIDER FACE validation set.
4. **Build model:** Build a face detector having the architecture of YOLOv1, train it using WIDERFACE WIDER FACE training set by splitting it into training set and validation set.

1 Detect objects in a face-free video using YOLOv8, YOLOv11, and YOLOv12 models. Compare their performance based on speed (FPS)

1.1 Dataset

A classroom video with **no human faces** was captured using a smartphone. Goal: Evaluate object detection capability of YOLOv8, YOLOv11, and YOLOv12.

1.2 Models Used

- YOLOv8n
- YOLOv11n
- YOLOv12n

1.3 Methodology

- Each model was loaded using the Ultralytics YOLO interface.
- The video was processed frame-by-frame.
- Detected objects were annotated on the frame.
- Frame processing time was recorded for FPS calculation.

1.4 Results

Model	FPS (Frames Per Second)
YOLOv8n	6.85
YOLOv11n	6.85
YOLOv12n	6.85

Table 1: Object detection speed comparison

Sample Output Frame:



Figure 1: Comparison of object detection using YOLOv8, YOLOv11, and YOLOv12 on a video without human faces.

1.5 Observations

- All models showed similar performance in speed.
- Objects like chairs, table, umbrellas and laptops were detected reliably.
- Demonstrates suitability for real-time inference in low-complexity environments.

2 Fine-Tuning YOLOv8 for Face Detection

2.1 Dataset

- **WIDER FACE dataset** (Training: 12,880 images, Validation: 3,226 images)
- Labels converted to YOLO format.

2.2 Training Configuration

- Model: YOLOv8n
- Image size: 640×640
- Batch size: 8
- Epochs: 5
- GPU: Tesla T4 (Google Colab)

2.3 Validation Results

Metric	Value
Precision	0.718
Recall	0.398
mAP@0.5	0.436
mAP@0.5:0.95	0.212
Inference Speed	2.8 ms
FPS	35.82

Table 2: Fine-tuned YOLOv8 face detection performance

Sample Output Image:

Google Colab Notebook

You can view the full implementation and results on Google Colab at the following link:

[Click here to open the Colab Notebook](#)

3 Comparison with YOLOv8-Face GitHub Model

3.1 Reference Model

- Source: YOLOv8-Face GitHub Repository

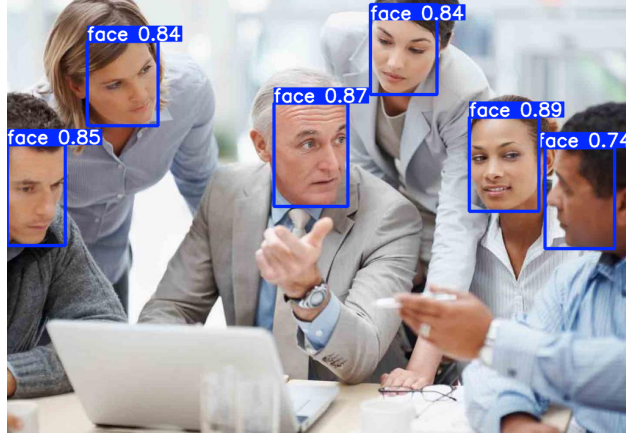


Figure 2: Sample face detection result using fine-tuned YOLOv8 model on the WIDER FACE dataset

- Pre-trained model optimized on WIDER FACE dataset.

3.2 Validation Results Comparison

Metric	Fine-Tuned YOLOv8	GitHub YOLOv8-Face
Precision	0.718	0.828
Recall	0.398	0.560
mAP@0.5	0.436	0.630
mAP@0.5:0.95	0.212	0.336
FPS	35.82	52.20

Table 3: Performance comparison of face detectors

3.3 Observations

- GitHub model outperforms the fine-tuned model in all evaluation metrics.
- Higher accuracy and faster FPS make the GitHub model more robust for deployment.
- Likely trained with more data and epochs or uses model-specific optimizations.

Google Colab Notebook

You can view the full implementation and results on Google Colab at the following link:

[Click here to open the Colab Notebook](#)

4 Build a face detector having the architecture of YOLOv1, train it using WIDERFACE WIDER FACE training set by splitting it into training set and validation set.

4.1 Dataset Details

The WIDER FACE dataset used in this problem.

Total Training Samples: 12,876

Total Validation Samples: 3,222

4.2 Model Architecture

The YOLOv1-inspired model was implemented using TensorFlow and designed to be memory-efficient while preserving the core YOLO structure.

Architecture Overview

```
Input (448×448×3)
→ Conv(32, 7×7, stride=2, LeakyReLU)
→ MaxPooling(2×2)
→ Conv(64, 3×3, LeakyReLU)
→ MaxPooling(2×2)
→ Conv(128, 3×3, LeakyReLU)
→ Conv(64, 1×1, LeakyReLU)
→ Conv(128, 3×3, LeakyReLU)
→ MaxPooling(2×2)
→ Conv(256, 3×3, LeakyReLU)
→ Conv(128, 1×1, LeakyReLU)
→ Conv(256, 3×3, LeakyReLU)
→ MaxPooling(2×2)
→ Conv(512, 3×3, LeakyReLU)
→ Conv(256, 1×1, LeakyReLU)
→ Flatten
→ Dense(512, LeakyReLU)
→ Dropout(0.5)
→ Dense(7×7×(2×5 + 1)) → Output (7×7×11)
```

This output tensor corresponds to:

- 7×7 grid cells ($S = 7$)
- 2 bounding boxes per cell ($B = 2$)
- 1 class ($C = 1$ for “face”)

5 Training Configuration

- **Optimizer:** Adam
- **Learning Rate:** $1e-4$
- **Loss Function:** Custom YOLOv1 loss
- **Batch Size:** 4
- **Epochs:** 30
- **Callbacks:**
 - ModelCheckpoint (save best model)
 - ReduceLROnPlateau (reduce LR on plateau)
 - EarlyStopping (patience = 5)

GPU memory growth was enabled to prevent out-of-memory errors.

6 Training Output

Samples Summary:

Training samples: 12876

Validation samples: 3222

Training History: A training history plot was generated showing the training and validation loss across epochs.

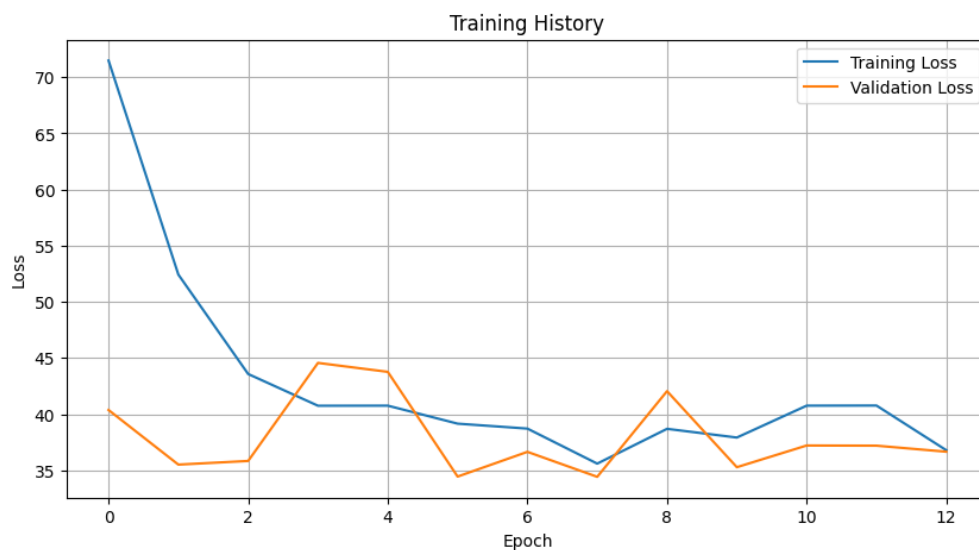


Figure 3: Training and Validation Loss Curve

7 Testing and Visualization

The trained YOLOv1 model was tested on sample validation images. Predicted bounding boxes were drawn on detected faces using OpenCV. Each box also displayed the confidence score.

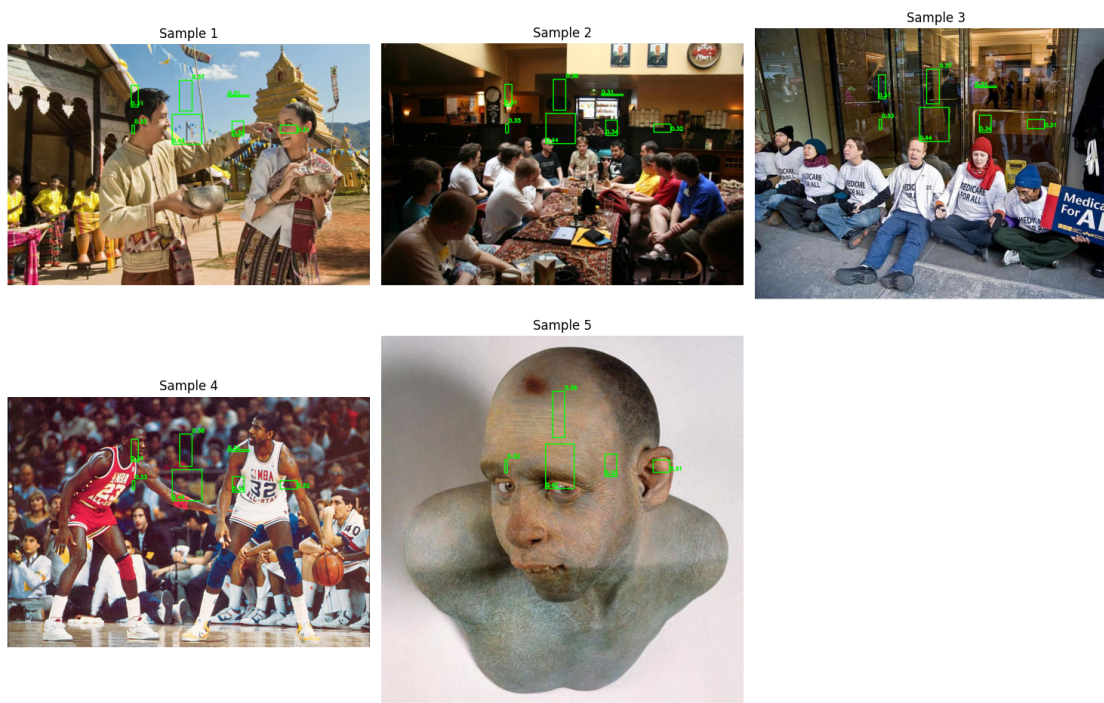


Figure 4: Sample Face Detection Results on Validation Images

8 Observation

The YOLOv1 model failed to draw proper bounding boxes around faces. Most detections were inaccurate or missing, indicating that the model did not learn face localization effectively. This suggests underfitting or insufficient training, and performance could improve with more epochs, better anchor tuning, or additional data augmentation.